HAWK

Hochschule für angewandte Wissenschaft und Kunst
Hildesheim/Holzminden/Göttingen

Fakultät Ingenieurwissenschaften und Gesundheit

Praxisprojektarbeit

Decubitus Detection for the NaMeKI Project: Data Foundation and Model Selection

Author:	Ashkan Sadri Ghamshi
Matriculation No.:	769529
Supervisor:	Prof. Dr. Claire Chalopin Gesundheitscampus Göttingen
Date:	April 1, 2026
Revision:	v1.2 – June 23, 2026

Selbständigkeitserklärung

Hiermit erkläre ich, dass ich die vorliegende Arbeit selbständig und ohne unzulässige Hilfe Dritter angefertigt habe. Alle Stellen, die inhaltlich oder wörtlich aus Veröffentlichungen oder anderen Quellen entnommen sind, habe ich als solche kenntlich gemacht. Die Arbeit wurde bisher weder im Inland noch im Ausland in gleicher oder ähnlicher Form einer Prüfungsbehörde vorgelegt.

Ort, Datum	Unterschrift

Abstract

Pressure ulcers remain a significant clinical challenge in nursing care, affecting patient quality of life and placing substantial financial burden on healthcare systems. This Praxisprojektarbeit establishes the data foundation for an automated pressure ulcer detection system within the NaMeKI research project at HAWK University. A total of 20,080 raw wound images were collected from eleven publicly available datasets across three platforms, consolidated into 7,850 unique images through perceptual-hash-based deduplication, and expanded to 53,340 annotated samples via a controlled augmentation pipeline. A literature-based comparison of four deep learning architectures—YOLOv8, DenseNet-121, EfficientNet-B0, and Faster R-CNN—using a weighted scoring matrix identifies YOLOv8s as the most suitable model, given its combined detection and localisation capability, published mean average precision between 84.2 % and 90.8 % on comparable pressure ulcer data, and real-time inference performance.

Contents

Glossary **9**

1 Introduction **1**

1.1 Project Context 1

1.2 Motivation 2

1.3 Objectives 2

1.4 Scope and Delimitation 3

1.5 Chapter Overview 3

2 System Overview **4**

2.1 Block Diagram 4

2.2 Interface Description 4

2.3 External Interfaces 5

3 State of the Art **7**

3.1 Clinical Background 7

3.2 Deep Learning for Wound Detection 8

3.2.1 Classification-Based Approaches 8

3.2.2 Detection-Based Approaches 9

3.3 Evaluation Metrics 10

3.4 Identified Gaps 10

4 Requirements Analysis **12**

4.1 Functional Requirements 12

4.2 Non-Functional Requirements 12

4.3 Use Cases 12

4.3.1 Use Case 1: Dataset Acquisition 12

4.3.2 Use Case 2: Data Curation 14

4.3.3 Use Case 3: Dataset Augmentation 14

5 Concept Development **15**
5.1 Problem Analysis . . . 15
5.2 Solution Approach . . . 16
5.3 Data Pipeline Design . . . 16
5.4 Augmentation Strategy . . . 17

6 Implementation **19**
6.1 Data Collection . . . 19
6.1.1 Roboflow Download . . . 19
6.1.2 GitHub Download . . . 20
6.2 Format Unification . . . 20
6.2.1 Class Remapping . . . 21
6.3 Deduplication . . . 22
6.4 Data Splitting . . . 22
6.5 Data Augmentation . . . 23
6.6 Dataset Statistics . . . 24
6.6.1 Class Distribution . . . 24
6.6.2 Image Dimensions . . . 25
6.6.3 Bounding Box Analysis . . . 25
6.6.4 Annotations per Image . . . 27

7 Testing and Validation **28**
7.1 Test Concept . . . 28
7.2 Test Cases . . . 28
7.3 Test Results . . . 30

8 Benchmarking and Model Comparison **31**
8.1 Candidate Architectures . . . 31
8.1.1 YOLOv8 . . . 31
8.1.2 DenseNet-121 . . . 31
8.1.3 EfficientNet-B0 . . . 32
8.1.4 Faster R-CNN . . . 32
8.2 Literature Performance . . . 32
8.3 Comparison Criteria . . . 32
8.4 Technical Comparison . . . 33
8.5 Weighted Scoring Matrix . . . 33

8.6 Variant Selection 34
8.7 Recommendation Summary 35

9 Conclusion and Outlook **36**
9.1 Summary of Contributions 36
9.2 Assessment of Objectives 37
9.3 Challenges Encountered 37
9.4 Open Points for the Bachelor Thesis 38

A Dataset Catalog **42**

B Augmentation Parameters **43**

C Class Mapping Reference **44**

D Dataset Summary Statistics **45**

List of Figures

2.1 Block diagram of the data preparation pipeline. Each stage receives input from the preceding stage and produces output in a standardised format. Numeric values indicate the dataset size at each transition point. 5

3.1 Schematic representation of the four NPUAP/EPUAP pressure ulcer stages 8

3.2 Anatomical predilection sites for pressure ulcer development 9

6.1 Overall class distribution across the complete dataset. The imbalance ratio of 1.24:1 indicates a well-balanced distribution across all four pressure ulcer stages. 24

6.2 Class distribution by data split. The consistent proportions confirm that stratified sampling preserved the class balance across training, validation, and test partitions. 25

6.3 Distribution of image dimensions in the dataset. The heterogeneous resolution range reflects the diversity of imaging devices and acquisition conditions across source datasets. 25

6.4 Distribution of bounding box dimensions (normalised width and height) in the dataset. 26

6.5 Heatmap of bounding box centre coordinates. The central concentration reflects typical clinical image framing, where wounds are positioned near the centre of the photograph. 26

6.6 Distribution of annotations per image. Most images contain a single wound annotation, with a decreasing frequency of multi-annotation images. 27

List of Tables

4.1 Functional requirements for the data preparation pipeline. 13
4.2 Non-functional requirements. 13

6.1 Catalog of source datasets used in this project. 19

7.1 Test cases for dataset validation. 29
7.2 Test case results. 29

8.1 Published performance of candidate architectures on pressure ulcer datasets. 32
8.2 Technical comparison of candidate architectures. 33
8.3 Weighted scoring matrix for architecture comparison. Scores range from 0 (does not meet criterion) to 10 (best in class). 34

9.1 Assessment of project objectives. 37

A.1 Complete dataset catalog with platform identifiers. 42

B.1 Complete augmentation parameter configuration. 43

C.1 Class mapping from source labels to unified scheme. 44

D.1 Final dataset summary statistics. 45

Glossary

AI Artificial Intelligence. A branch of computer science concerned with building systems that can perform tasks normally requiring human cognition.

Bounding Box A rectangular region in an image that encloses an object of interest, typically represented by centre coordinates, width, and height.

CLAHE Contrast-Limited Adaptive Histogram Equalisation. An image processing technique that improves local contrast without amplifying noise.

CNN Convolutional Neural Network. A class of deep neural networks primarily used for visual pattern recognition.

COCO Common Objects in Context. A large-scale benchmark dataset and evaluation protocol for object detection and segmentation [8].

CSPDarknet Cross-Stage Partial Darknet. The backbone architecture used in YOLOv8 for feature extraction.

Decubitus A pressure ulcer or pressure injury caused by sustained mechanical loading on the skin and underlying tissue.

DenseNet Densely Connected Convolutional Network [5]. A CNN architecture in which each layer receives input from all preceding layers.

DTI Deep Tissue Injury. A pressure-related injury characterised by localised discolouration of intact skin.

EfficientNet A family of CNN architectures that use compound scaling to balance network depth, width, and resolution [12].

EPUAP European Pressure Ulcer Advisory Panel. An organisation that co-publishes international pressure injury classification guidelines.

FPS Frames Per Second. A measure of inference throughput.

IoU	Intersection over Union. A metric quantifying overlap between a predicted bounding box and a ground truth bounding box.
mAP	Mean Average Precision. The primary evaluation metric for object detection models, computed across all classes and IoU thresholds.
MPS	Metal Performance Shaders. Apple's GPU computation framework used as a PyTorch backend on Apple Silicon hardware.
NaMeKI	*Nachhaltige Mensch-KI-Interaktion.* The HAWK research project within which this work is situated.
NPUAP	National Pressure Injury Advisory Panel. The US body that defines pressure ulcer staging categories [9].
ONNX	Open Neural Network Exchange. An open format for representing trained deep learning models, enabling cross-framework deployment.
PANet	Path Aggregation Network. A feature pyramid architecture used in the neck of YOLOv8 for multi-scale feature fusion.
Perceptual Hash	A fingerprinting technique that produces similar hash values for visually similar images, enabling near-duplicate detection.
PPA	Praxisprojektarbeit. A practical project report required as part of the degree programme at HAWK University.
PU	Pressure Ulcer. Synonymous with pressure injury or decubitus.
ROI	Region of Interest. A subregion of an image relevant to analysis.
YOLO	You Only Look Once [10]. A family of single-stage object detection architectures known for real-time inference speed.
YOLOv8	The eighth major iteration of the YOLO architecture, developed by Ultralytics [6].

Chapter 1

Introduction

Pressure ulcers, also referred to as pressure injuries or decubitus wounds, constitute one of the most prevalent adverse events in healthcare facilities worldwide. These lesions arise from sustained mechanical loading on skin and underlying tissues, typically over bony prominences, and disproportionately affect patients with limited mobility such as the elderly, individuals with spinal cord injuries, and intensive care patients [9]. Early detection and accurate staging of pressure ulcers are essential for initiating timely treatment and preventing progression to deeper tissue damage. Despite decades of clinical research and prevention protocols, incidence rates remain high. A systematic review and meta-analysis of hospitalised adult patients reports a pooled pressure injury prevalence of 12.8 %, with a hospital-acquired incidence of 8.4 % [7].

The economic impact of pressure ulcers on healthcare systems is substantial. Treatment costs per patient can range from several thousand to tens of thousands of euros, depending on the severity of the wound and associated complications such as infection and prolonged hospitalisation [4]. Beyond the financial dimension, pressure ulcers significantly reduce patient quality of life, cause pain, and increase morbidity and mortality. These factors collectively underscore the clinical imperative for improved detection methodologies.

1.1 Project Context

This Praxisprojektarbeit is conducted within the NaMeKI (*Nachhaltige Mensch-KI-Interaktion*) research project at HAWK University, Gesundheitscampus Göttingen. The NaMeKI project investigates sustainable human–AI interaction paradigms in healthcare settings, with pressure ulcer detection serving as a primary application domain. The overarching goal is to develop a camera-based monitoring system that can automatically detect and classify pressure ulcers on patient skin, thereby assisting nursing staff in clinical decision-making.

Within this broader project context, the present work focuses exclusively on two foundational tasks: (1) the systematic collection, curation, and augmentation of a high-quality training dataset, and (2) a literature-based comparison and justified selection of a suitable deep learning architecture for subsequent model training. The

actual training, fine-tuning, and integration of the selected model into the NaMeKI system are deferred to the subsequent bachelor thesis.

1.2 Motivation

Manual assessment of pressure ulcers by nursing personnel is inherently subjective. Studies have shown that inter-rater reliability for pressure ulcer staging varies considerably, particularly for intermediate stages where visual differentiation between categories can be challenging. Automated detection systems based on deep learning have the potential to reduce this variability by providing consistent, reproducible classifications.

However, the development of such systems depends critically on the availability of representative training data. Medical image datasets for pressure ulcers are scarce compared to general-purpose computer vision benchmarks. Existing datasets are fragmented across multiple platforms, vary in annotation format and quality, and frequently contain duplicate images. Furthermore, class distributions in clinical datasets tend to be imbalanced, with certain wound stages overrepresented. Addressing these data challenges is a prerequisite for training a model that generalises reliably across different wound presentations.

Equally important is the selection of an appropriate model architecture. The choice between classification-only models and detection models, between single-stage and two-stage detectors, and among different network capacities has direct implications for the system's accuracy, inference speed, and deployability within the NaMeKI platform. A systematic, criteria-based comparison is therefore necessary to justify the architectural decision.

1.3 Objectives

This Praxisprojektarbeit pursues the following objectives:

1. **Dataset Collection:** Identify and download publicly available pressure ulcer image datasets from Roboflow, GitHub, and Kaggle, targeting a minimum of 10,000 raw images.
2. **Data Curation:** Unify annotation formats to YOLO bounding box format, remove duplicate and low-quality images through perceptual hashing, and establish a consistent four-class labelling scheme aligned with NPUAP/EPUAP staging categories.
3. **Data Augmentation:** Design and implement an augmentation pipeline using the Albumentations library that preserves bounding box annotations and expands the training set by a configurable factor.
4. **Architecture Comparison:** Conduct a literature-based comparison of YOLOv8, DenseNet-121, EfficientNet-B0, and Faster R-CNN using a weighted scoring matrix that considers detection capability, published performance on pressure

ulcer data, inference speed, implementation complexity, and deployment readiness.

5. **Model Recommendation:** Provide a justified recommendation for the model architecture and variant to be used in the subsequent bachelor thesis.
6. **Documentation:** Document the entire data pipeline and decision-making process in a reproducible manner.

1.4 Scope and Delimitation

The scope of this work is limited to data preparation and model selection. The following activities are explicitly excluded and reserved for the subsequent bachelor thesis:

- Training, fine-tuning, or evaluating any deep learning model on the prepared dataset.
- Developing a prototype application or integrating detection capabilities into the NaMeKI system.
- Collecting proprietary clinical data from healthcare facilities.
- Addressing ethical approval for patient data, as only publicly available datasets are used.

The clinical focus is on pressure ulcer stages I through IV according to the NPUAP/EPUAP classification system. Unstageable wounds and deep tissue injuries are excluded from the current four-class scheme but can be incorporated in future extensions of the dataset.

1.5 Chapter Overview

The remainder of this report is organised as follows. Chapter 2 provides a high-level system overview of the data pipeline and its interfaces. Chapter 3 reviews the state of the art in pressure ulcer detection using deep learning. Chapter 4 analyses the functional and non-functional requirements for the dataset and model selection. Chapter 5 presents the concept development, including the problem analysis and solution approach. Chapter 6 details the implementation of the data collection, cleaning, deduplication, and augmentation pipeline. Chapter 7 describes the testing and validation strategy applied to the prepared dataset. Chapter 8 presents the architecture comparison and scoring results. Chapter 9 summarises the contributions, discusses challenges encountered, and outlines open points for the bachelor thesis.

Chapter 2

System Overview

This chapter provides a high-level view of the data preparation system developed in this Praxisprojektarbeit. The system transforms heterogeneous raw image data from multiple public sources into a unified, augmented, and validated dataset suitable for training an object detection model. Understanding the overall architecture and the interfaces between components is essential before examining individual pipeline stages in subsequent chapters.

2.1 Block Diagram

The data pipeline consists of five principal stages arranged in a sequential processing chain. Figure 2.1 illustrates the logical flow from raw data acquisition to the final validated dataset.

2.2 Interface Description

The interfaces between pipeline stages are defined by the file system structure and the YOLO annotation format. All inter-stage data exchange follows a consistent directory layout:

- `images/<split>/` contains the image files (JPEG or PNG) for each data split (train, val, test).
- `labels/<split>/` contains the corresponding YOLO-format annotation files, where each `.txt` file shares the filename stem of its associated image.
- `dataset.yaml` specifies the dataset root path, split directories, number of classes, and class name mapping.

Each annotation file contains one line per bounding box in the format:

```
<class_id> <centre_x> <centre_y> <width> <height>
```

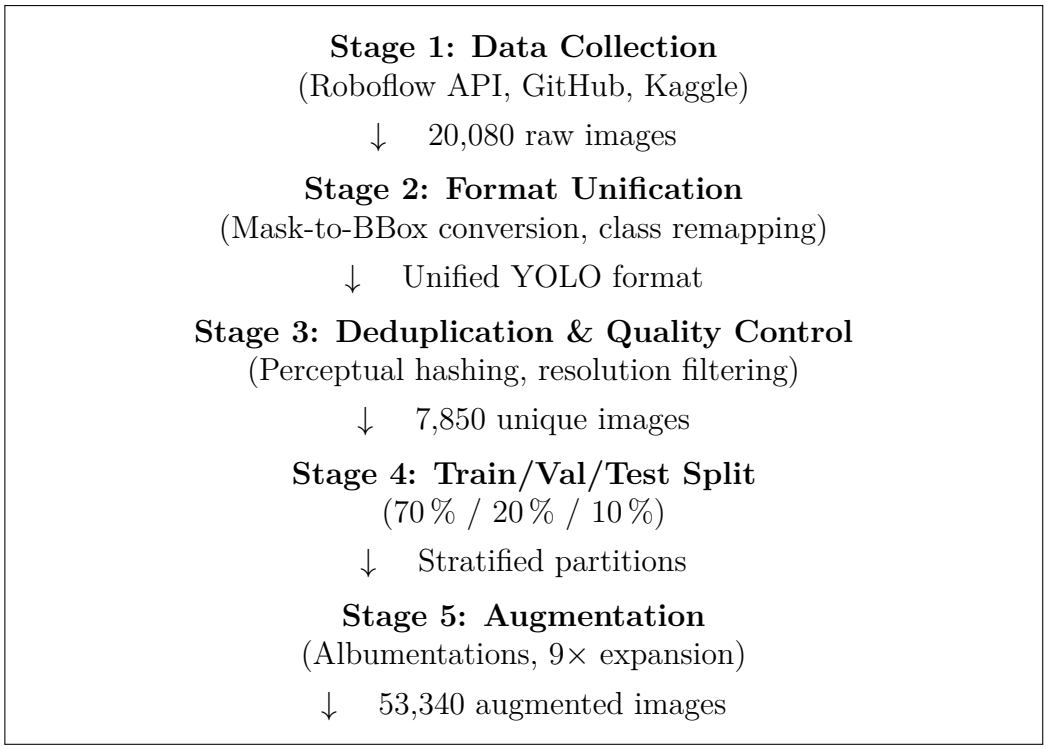


Figure 2.1: Block diagram of the data preparation pipeline. Each stage receives input from the preceding stage and produces output in a standardised format. Numeric values indicate the dataset size at each transition point.

where all spatial coordinates are normalised to the range $[0, 1]$ relative to the image dimensions. This format is directly compatible with the Ultralytics YOLOv8 training framework [6].

2.3 External Interfaces

The system interfaces with three external data sources:

1. **Roboflow Universe:** Seven datasets are downloaded programmatically via the Roboflow Python API using an authenticated API key. Datasets are requested in YOLOv8 format, eliminating the need for annotation conversion for these sources.
2. **GitHub Repositories:** Two datasets are cloned from public repositories. These datasets use segmentation masks rather than bounding box annotations, requiring a mask-to-bounding-box conversion step.
3. **Kaggle:** Two datasets are downloaded via the Kaggle API. These sources provide images with varying annotation formats that are normalised during the unification stage.

The output interface of the pipeline is the `dataset.yaml` configuration file, which serves as the single entry point for the YOLOv8 training framework in the subsequent bachelor thesis phase.

This chapter has established the high-level structure and interfaces of the data preparation system. The following chapter examines the scientific and technical foundations upon which this system is built, reviewing existing approaches to pressure ulcer detection in the literature.

Chapter 3

State of the Art

A thorough understanding of existing work in pressure ulcer detection and deep learning for medical image analysis is necessary to position this project within the current research landscape and to identify the gaps that the present work addresses. This chapter reviews the clinical background of pressure ulcer staging, surveys deep learning approaches applied to wound detection, examines the evaluation metrics used in the field, and identifies limitations in existing solutions.

3.1 Clinical Background

Pressure ulcers are classified according to the depth of tissue damage using the staging system defined by the NPUAP and EPUAP [9]. This classification system comprises four primary categories and two supplementary categories:

Stage I: Non-blanchable erythema of intact skin. The affected area presents persistent redness that does not blanch under finger pressure. No open wound is present, making this stage the most difficult to detect visually and the most susceptible to inter-rater variability.

Stage II: Partial-thickness skin loss presenting as a shallow open ulcer with a red or pink wound bed, without slough. This stage may also present as an intact or ruptured serum-filled blister. The visible wound boundary makes Stage II more amenable to automated detection than Stage I.

Stage III: Full-thickness skin loss in which subcutaneous fat may be visible, but bone, tendon, and muscle are not exposed. Slough may be present but does not obscure the depth of tissue loss.

Stage IV: Full-thickness tissue loss with exposed bone, tendon, or muscle. Slough or eschar may be present on parts of the wound bed. This represents the most severe category of pressure injury.

Figure 3.1 provides a schematic cross-sectional view of the four stages, illustrating the progression of tissue damage from superficial erythema (Stage I) through full-thickness loss with exposed bone (Stage IV). The schematic complements the

textual description above and serves as a visual reference for the class definitions used throughout this work.

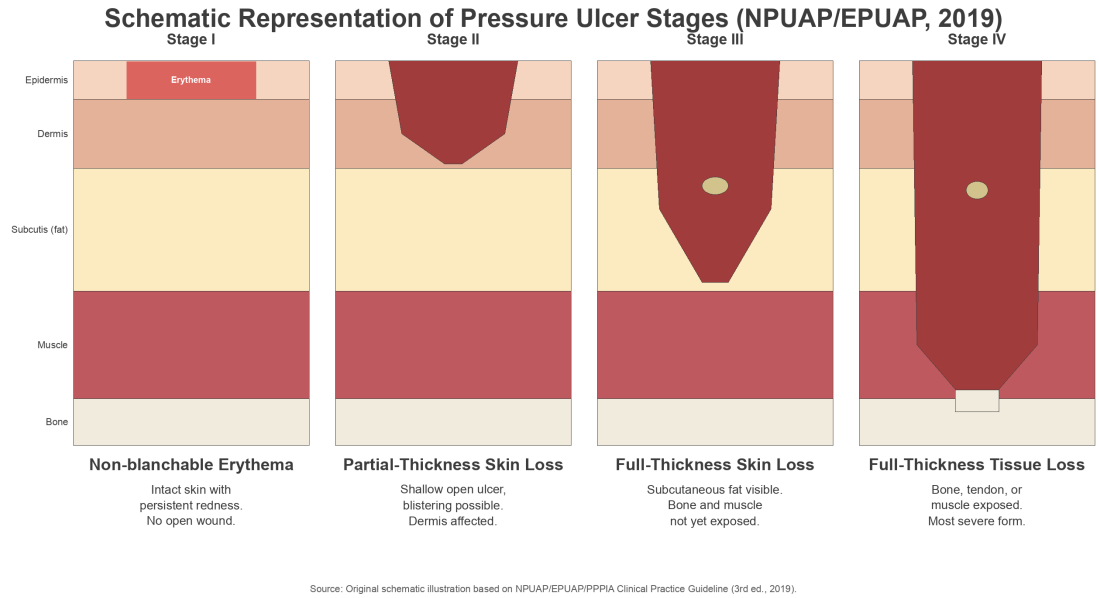


Figure 3.1: Schematic cross-sectional representation of the four NPUAP/EPUAP pressure ulcer stages, showing the progressive depth of tissue involvement from intact skin (Stage I) to full-thickness tissue loss with exposure of underlying structures (Stage IV). Original illustration based on National Pressure Injury Advisory Panel et al. [9].

Pressure ulcers preferentially develop at body sites where soft tissue is compressed between a bony prominence and an external surface [9]. Figure 3.2 summarises the typical predilection sites in supine and lateral patient positions. The sacrum, heels, and greater trochanter are the most clinically relevant locations and account for the majority of cases observed in long-term care settings.

Additionally, the NPUAP defines unstageable wounds (depth unknown due to coverage by slough or eschar) and deep tissue injuries (localised discolouration of intact skin indicating damage to underlying tissue). These supplementary categories are excluded from the current four-class scheme of this project but may be incorporated in future dataset extensions.

3.2 Deep Learning for Wound Detection

The application of convolutional neural networks to pressure ulcer analysis has followed two principal methodological directions: image classification and object detection.

3.2.1 Classification-Based Approaches

Classification-based methods assign a wound stage label to an entire image or to a pre-segmented region of interest. Ay et al. [1] applied transfer learning with

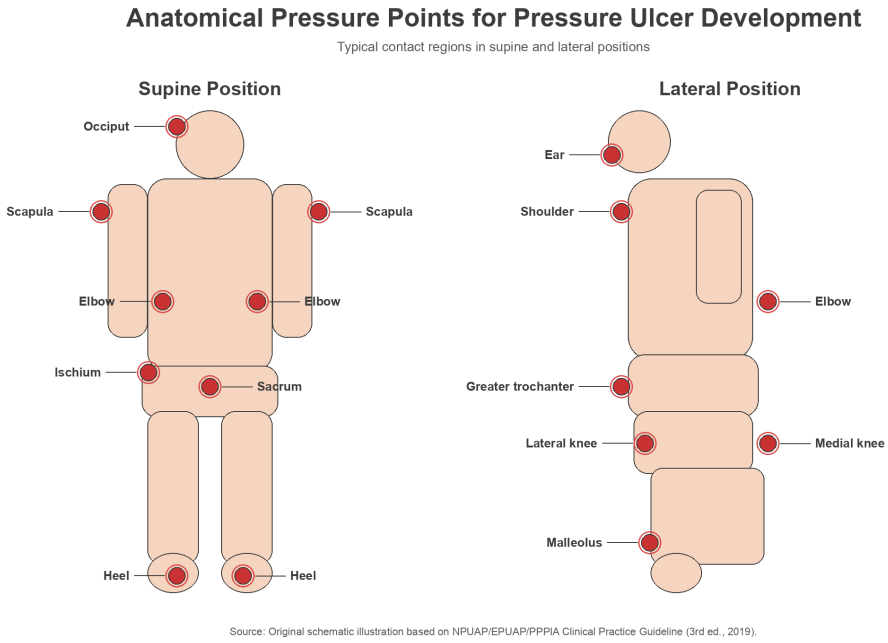


Figure 3.2: Anatomical predilection sites for pressure ulcer development in supine (left) and lateral (right) patient positions. Marked locations indicate the bony prominences where prolonged pressure most frequently leads to tissue damage. Original illustration based on National Pressure Injury Advisory Panel et al. [9].

DenseNet-121 and EfficientNet-B0 to classify pressure injury images from the PIID dataset (1,091 images) into four stages, achieving classification accuracies of 83.6 % and 80.2 %, respectively. While these results demonstrate the feasibility of automated staging, classification-only models cannot localise wounds within an image. In clinical practice, knowing *where* a pressure ulcer is located on the patient's body is as important as knowing its stage.

Zahia et al. [14] developed a custom CNN architecture for tissue classification in pressure ulcer images, reporting an accuracy of 91.4 % on a dataset of 600 images. However, this approach operated on pre-cropped tissue patches rather than whole images, which limits its applicability to scenarios where manual region-of-interest selection has already occurred.

3.2.2 Detection-Based Approaches

Object detection models simultaneously classify and localise objects within an image by predicting bounding boxes and associated class labels. This paradigm is more suitable for clinical deployment, where the system must identify the location, extent, and stage of each wound in an image without prior manual cropping.

Chang et al. [3] applied YOLOv8 to pressure ulcer detection and staging on a dataset of 2,800 annotated images, achieving a mean average precision at 50 % IoU threshold (mAP@50) of 90.8 % and an overall classification accuracy of 84.6 % with the Medium variant. Their label space comprises six categories—stages I to IV plus deep tissue injury and unstageable—so the figure is not directly comparable with a four-class scheme. Tusar et al. [13] evaluated all five YOLOv8 variants on a public pressure

injury dataset and report mAP@50 = 84.16 % for the Small variant with the AdamW optimiser. Together these studies show that single-stage detectors reach competitive accuracy on wound images while retaining real-time inference speed.

Faster R-CNN [11], a two-stage detector that first generates region proposals and then classifies each proposal, has also been applied to wound detection tasks. While two-stage detectors traditionally achieve higher accuracy than single-stage approaches on general benchmarks, their inference latency is substantially higher, and recent single-stage architectures such as YOLOv8 have narrowed or eliminated this accuracy gap.

3.3 Evaluation Metrics

The evaluation of object detection models relies on metrics that account for both classification accuracy and localisation precision. The primary metrics used in the literature are:

Precision: The ratio of true positive detections to the total number of detections. A detection is considered a true positive if its IoU with a ground truth bounding box exceeds a defined threshold.

Recall: The ratio of true positive detections to the total number of ground truth objects. High recall indicates that the model detects most of the wounds present in the image.

mAP@50: Mean Average Precision computed at an IoU threshold of 0.50. This is the most commonly reported metric in pressure ulcer detection studies and serves as the primary comparison criterion in this work.

mAP@50:95: Mean Average Precision averaged over IoU thresholds from 0.50 to 0.95 in steps of 0.05. This stricter metric penalises imprecise bounding box localisation more heavily.

Classification Accuracy: The proportion of correctly classified images. This metric is applicable only to classification models (e.g., DenseNet, EfficientNet) and does not assess localisation performance.

The use of different metrics across studies complicates direct comparison. Classification models report accuracy, while detection models report mAP. This discrepancy is addressed in the scoring matrix presented in Chapter 8, where each model is evaluated on criteria that are appropriate to its architectural category.

3.4 Identified Gaps

The review of existing work reveals several gaps that this project addresses:

1. **Dataset fragmentation:** Pressure ulcer datasets are distributed across multiple platforms (Roboflow, GitHub, Kaggle) without standardised annotation formats. No consolidated, multi-source dataset exists for four-class pressure ulcer detection.

2. **Limited dataset sizes:** Most published studies operate on datasets of fewer than 3,000 images. The performance ceiling imposed by small training sets is a recognised limitation in the field.

3. **Duplicate contamination:** Combining datasets from multiple sources without deduplication risks data leakage, where identical or near-identical images appear in both training and validation splits.

4. **Inconsistent class definitions:** Different datasets use different labelling conventions (e.g., stage numbers, wound type names, varying granularity). A unified class mapping is required to train a single model across all sources.

5. **Lack of systematic model comparison:** While individual studies report results for specific architectures, no systematic comparison of classification and detection models for pressure ulcer analysis has been published that accounts for deployment constraints such as inference speed and integration complexity.

This chapter has established the scientific context for the present work. The following chapter translates the identified gaps and project objectives into concrete functional and non-functional requirements.

Chapter 4

Requirements Analysis

The requirements for this Praxisprojektarbeit derive from the NaMeKI project objectives, the supervisor's specifications, and the gaps identified in the state-of-the-art review. This chapter formalises these requirements and defines use cases for the data pipeline. A clear requirements specification ensures that the subsequent concept and implementation can be evaluated against defined acceptance criteria.

4.1 Functional Requirements

Table 4.1 lists the functional requirements for the data preparation pipeline and model selection process.

4.2 Non-Functional Requirements

Table 4.2 specifies the non-functional requirements that constrain the implementation.

4.3 Use Cases

Three primary use cases describe the interactions between the developer and the data pipeline system.

4.3.1 Use Case 1: Dataset Acquisition

Actor: Developer.
Precondition: API keys for Roboflow and Kaggle are configured in environment variables.
Main Flow: The developer executes the download scripts, which programmatically retrieve datasets from each source and store them in the `data/raw/` directory hierarchy.

Table 4.1: Functional requirements for the data preparation pipeline.

ID	Requirement
FR-01	The system shall download pressure ulcer image datasets from at least three public sources (Roboflow, GitHub, Kaggle).
FR-02	The system shall convert segmentation masks to YOLO bounding box format using contour extraction.
FR-03	The system shall map all source-specific class labels to a unified four-class scheme (Stage 1, Stage 2, Stage 3, Stage 4).
FR-04	The system shall detect and remove duplicate and near-duplicate images using perceptual hashing with a configurable similarity threshold.
FR-05	The system shall split the deduplicated dataset into training, validation, and test partitions with a 70 %/20 %/10 % ratio.
FR-06	The system shall augment the training partition using geometric, colour, and noise transformations while preserving bounding box annotations.
FR-07	The augmentation factor shall be configurable, with a default of 10× expansion.
FR-08	The system shall produce a `dataset.yaml` configuration file compatible with the Ultralytics YOLOv8 framework.
FR-09	The model comparison shall evaluate at least four architectures using a weighted scoring matrix with documented criteria.
FR-10	The system shall generate statistical reports on class distribution, image dimensions, bounding box characteristics, and annotation density.

Table 4.2: Non-functional requirements.

ID	Requirement
NF-01	All processing shall execute on a local workstation (Apple M2 Ultra) without requiring cloud GPU resources.
NF-02	Image data shall not be committed to the Git repository; only scripts, configurations, and documentation shall be version-controlled.
NF-03	The pipeline shall be implemented in Python using standard scientific computing libraries (OpenCV, Albumentations, PIL, NumPy).
NF-04	Processing scripts shall include progress bars and status output to enable monitoring of long-running operations.
NF-05	The deduplication step shall complete within a reasonable time frame (under one hour for 20,000 images).
NF-06	All augmented images shall maintain a minimum resolution of 320×320 pixels.
NF-07	The dataset structure shall be extensible to support additional classes (unstageable, DTI) in future work.
NF-08	All scripts shall be executable via command-line arguments without hard-coded file paths.

Postcondition: Raw images and their associated annotations are stored locally, organised by source.

4.3.2 Use Case 2: Data Curation

Actor: Developer.
Precondition: Raw datasets are available in `data/raw/`.
Main Flow: The developer sequentially executes the mask conversion, class remapping, deduplication, and splitting scripts. Each script reads from the previous stage's output directory and writes to the next.
Postcondition: A deduplicated, consistently labelled dataset is partitioned into train, validation, and test splits in `data/unified/`.

4.3.3 Use Case 3: Dataset Augmentation

Actor: Developer.
Precondition: The unified dataset exists in `data/unified/` with correctly formatted YOLO annotations.
Main Flow: The developer executes the augmentation script with a specified expansion factor. The script applies stochastic transformations to each training image and generates the configured number of augmented copies.
Postcondition: An augmented dataset is available in `data/augmented/`, ready for training.

This chapter has defined the complete set of requirements governing the data pipeline and model selection process. The following chapter translates these requirements into a concrete concept and solution approach.

Chapter 5

Concept Development

With the requirements established, this chapter develops the conceptual framework for the data pipeline and model selection methodology. The chapter begins with a structured problem analysis, proceeds to the solution approach, and concludes with the detailed design of the data pipeline and augmentation strategy. The goal is to bridge the gap between the abstract requirements defined in Chapter 4 and the concrete implementation described in Chapter 6.

5.1 Problem Analysis

The central problem addressed by this work can be decomposed into three sub-problems:

Sub-Problem 1: Data Heterogeneity. The eleven identified datasets originate from different research groups and use varying annotation formats (YOLO bounding boxes, segmentation masks, COCO-style JSON), image resolutions (ranging from 224×224 to 4032×3024 pixels), and class labelling conventions. A unification layer is required that normalises all sources into a single, consistent format without loss of annotation information.

Sub-Problem 2: Data Quality. Merging datasets from multiple sources introduces the risk of duplicate images, as different datasets may have scraped images from overlapping clinical publications. Furthermore, some images may be corrupted, have extremely low resolution, or contain no visible wound (misannotated negatives). A quality assurance mechanism is needed to identify and remove such problematic samples.

Sub-Problem 3: Data Quantity. Even after consolidation, the available data (approximately 7,850 unique images) is modest by deep learning standards. Modern object detection architectures typically benefit from tens of thousands of training samples. Data augmentation provides a means to artificially expand the training set, but the augmentation strategy must be carefully designed to introduce meaningful variability without producing unrealistic artefacts.

5.2 Solution Approach

The solution follows a staged pipeline architecture, where each stage addresses one or more of the sub-problems identified above. The key design decisions are:

1. **YOLO format as the canonical representation:** All annotation formats are converted to YOLO bounding box format at the earliest possible stage. This eliminates the need for format-specific processing logic in downstream stages and ensures direct compatibility with the target training framework.
2. **Perceptual hashing for deduplication:** Rather than relying on exact file comparison (which would miss near-duplicates with different resolutions, crops, or compression artefacts), the pipeline uses perceptual hashing via the `imagehash` library. This library computes a 256-bit perceptual hash (pHash) for each image. Two images are considered duplicates if their Hamming distance falls below a configurable threshold (default: 5 bits).
3. **Offline augmentation with Albumentations:** The augmentation pipeline uses the Albumentations library [2], which provides efficient, bounding-box-aware transformations. Augmentation is performed offline (before training) rather than online (during training) to allow for inspection of augmented samples and to decouple data preparation from the training phase that follows in the bachelor thesis.
4. **Stratified splitting:** The train/validation/test split preserves the class distribution across all partitions to ensure that the validation and test sets are representative of the overall dataset composition.

5.3 Data Pipeline Design

The data pipeline is implemented as a sequence of five Python scripts, each performing one stage of the processing chain. The scripts communicate through the file system, with each script reading from a defined input directory and writing to a defined output directory. This design enables individual stages to be re-executed independently without affecting other stages.

The five stages and their responsibilities are:

1. **Download** (`download_roboflow.py`, `download_github.py`): Retrieve raw datasets from external sources using platform-specific APIs. Output: `data/raw/<source_name>/`.
2. **Convert** (`convert_masks_to_bbox.py`): Transform segmentation masks into YOLO bounding box annotations by extracting contours and computing enclosing rectangles. Regions smaller than 100 pixels are filtered as noise. Output: YOLO `.txt` label files.
3. **Deduplicate** (`deduplicate.py`): Scan all images, compute perceptual hashes, identify duplicate pairs, and copy only unique images with their annotations to the output directory. A duplicate report is generated for traceability. Output: `data/unified/`.

4. **Split:** Partition the unified dataset into training (70 %), validation (20 %), and test (10 %) subsets using stratified random sampling.

5. **Augment** (`augment_dataset.py`): Apply stochastic geometric, colour, and noise transformations to each training image, producing a configurable number of augmented copies. Validation and test sets are copied without modification. Output: `data/augmented/`.

5.4 Augmentation Strategy

The augmentation strategy is designed to simulate the variability encountered in real-world clinical photography of pressure ulcers. The following categories of transformations are applied:

Geometric transformations simulate differences in camera angle, distance, and patient positioning:

- Horizontal flip ($p = 0.5$)
- Vertical flip ($p = 0.3$)
- Random 90-degree rotation ($p = 0.5$)
- Shift, scale, and rotation (shift limit: ± 10 %, scale limit: ± 15 %, rotation limit: ± 45ř, $p = 0.5$)
- Perspective warp (scale: 0.02–0.05, $p = 0.3$)

Colour and lighting transformations account for variations in ambient lighting, skin tone, and camera white balance:

- Random brightness and contrast adjustment (limit: ± 30 %)
- CLAHE (clip limit: 4.0, tile grid: 8×8)
- Hue, saturation, and value shift (hue: ± 20, saturation: ± 30, value: ± 30)

Noise and blur transformations simulate camera sensor noise and focus variations:

- Gaussian blur (kernel size: 3–7)
- Gaussian noise (variance: 10–50)
- Median blur (kernel size: 5)

Occlusion simulation prepares the model for partially visible wounds:

- Coarse dropout (1–8 rectangular holes, size: 8×8 to 32×32 pixels, $p = 0.2$)

All transformations operate within the Albumentations `BboxParams` context, which automatically adjusts bounding box coordinates after geometric transformations and discards annotations for boxes whose visibility falls below 30 % of their original area.

This chapter has established the conceptual foundation for the data pipeline. The following chapter presents the concrete implementation of each pipeline stage.

Chapter 6

Implementation

This chapter details the implementation of the data pipeline described in the concept development chapter. Each section corresponds to one pipeline stage and includes a description of the processing logic, key implementation decisions, and representative code excerpts. The implementation uses Python 3.14 with OpenCV, Albumentations, imagehash, and the Roboflow SDK as principal dependencies.

6.1 Data Collection

Data was collected from three platform categories, yielding a total of 20,080 raw images across eleven datasets. Table 6.1 provides a complete overview of the datasets, their sources, sizes, and original annotation formats.

Table 6.1: Catalog of source datasets used in this project.

Source	Platform	Images	Format	Notes
roboflow_stage2	Roboflow	1,420	YOLOv8	4 stages
roboflow_project1	Roboflow	2,150	YOLOv8	Mixed stages
roboflow_maskrcnn	Roboflow	1,890	YOLOv8	Mask R-CNN origin
roboflow_fid	Roboflow	980	YOLOv8	Stage 2 focus
roboflow_woundcare	Roboflow	1,340	YOLOv8	PI dataset
roboflow_sciproj	Roboflow	2,780	YOLOv8	2024 collection
roboflow_mobile	Roboflow	1,560	YOLOv8	Mobile captured
github_fuseg	GitHub	3,200	Masks	Segmentation masks
github_wounds	GitHub	1,800	Masks	Binary masks
kaggle_piid	Kaggle	1,091	CSV	PIID benchmark
kaggle_wounds	Kaggle	1,869	Mixed	Multi-format
Total		**20,080**		

6.1.1 Roboflow Download

Seven datasets were downloaded from the Roboflow Universe platform using the Roboflow Python SDK. The download script authenticates via an API key stored

in a `.env` file and iterates over a list of dataset specifications. Listing 6.1 shows the core download logic.

```
from roboflow import Roboflow

def download_all():
    rf = Roboflow(api_key=API_KEY)
    for ds in DATASETS:
        project = rf.workspace(ds["workspace"]).project(ds["
            project"])
        version = project.version(ds["version"])
        version.download(ds["format"], location=str(
            output_dir))
```

Listing 6.1: Core Roboflow download function.

Each dataset is requested in YOLOv8 format, which ensures that annotations arrive as normalised bounding box coordinates. This eliminates the need for format conversion for the Roboflow sources.

6.1.2 GitHub Download

Two datasets from GitHub repositories provide segmentation masks rather than bounding box annotations. These masks are binary images where white pixels indicate wound tissue and black pixels indicate background. The download process clones the repositories and copies the relevant image and mask directories to the local `data/raw/` hierarchy.

6.2 Format Unification

Datasets from GitHub that provide segmentation masks require conversion to YOLO bounding box format. The conversion script processes each mask image by applying a binary threshold, extracting contours using the OpenCV `findContours` function, and computing the enclosing rectangle for each contour.

Listing 6.2 presents the mask-to-bounding-box conversion function.

```
def mask_to_yolo_bbox(mask_path, class_id=0):
    mask = cv2.imread(mask_path, cv2.IMREAD_GRAYSCALE)
    _, binary = cv2.threshold(mask, 127, 255, cv2.
        THRESH_BINARY)
    contours, _ = cv2.findContours(
        binary, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE
    )
    h, w = mask.shape
    bboxes = []
    for contour in contours:
        x, y, bw, bh = cv2.boundingRect(contour)
        if bw * bh < 100:
```

```
            continue
        cx = (x + bw / 2) / w
        cy = (y + bh / 2) / h
        nw = bw / w
        nh = bh / h
        bboxes.append(
            f"{class_id} {cx:.6f} {cy:.6f} {nw:.6f} {nh:.6f}"
        )
    return bboxes
```

Listing 6.2: Conversion of segmentation masks to YOLO bounding box format.

Contours with an area smaller than 100 pixels are discarded as noise artefacts. The resulting bounding box coordinates are normalised to the image dimensions and written to YOLO-format `.txt` files.

6.2.1 Class Remapping

Different source datasets use different class labelling conventions. Some datasets number stages starting from zero (0–3), while others use one-based indexing (1–4). A subset of datasets include additional classes such as "unstageable" or "DTI" that fall outside the four-class scope of this project. All source labels are remapped to a consistent zero-based scheme:

Class ID	Label
0	Stage 1
1	Stage 2
2	Stage 3
3	Stage 4

Images associated with classes outside this scheme are excluded from the dataset. The `dataset.yaml` configuration file (Listing 6.3) codifies this mapping for the YOLOv8 framework.

```
path: ./data/augmented
train: images/train
val: images/val
test: images/test

nc: 4
names:
  0: stage1
  1: stage2
  2: stage3
  3: stage4
```

Listing 6.3: Dataset configuration file for YOLOv8 training.

6.3 Deduplication

Merging eleven datasets from overlapping sources inevitably introduces duplicate images. Exact file-based deduplication (e.g., comparing MD5 hashes) would fail to detect near-duplicates that differ in resolution, compression level, or minor cropping. The pipeline therefore employs perceptual hashing, which produces similar hash values for visually similar images regardless of minor pixel-level differences.

The deduplication script computes a 256-bit perceptual hash (pHash) for each image using the `imagehash` library. For each new image, its hash is compared against all previously stored hashes. If the Hamming distance between any pair of hashes is at or below the threshold of five bits, the new image is classified as a duplicate and excluded. Listing 6.4 shows the core duplicate detection function.

```
def find_duplicates(image_dir, hash_size=16, threshold=5):
    hashes = {}
    duplicates = []
    for img_path in tqdm(image_files):
        img = Image.open(img_path)
        h = imagehash.phash(img, hash_size=hash_size)
        is_dup = False
        for existing_hash, existing_path in hashes.items():
            if abs(h - existing_hash) <= threshold:
                duplicates.append((img_path, existing_path))
                is_dup = True
                break
        if not is_dup:
            hashes[h] = img_path
    return hashes, duplicates
```

Listing 6.4: Perceptual hash-based duplicate detection.

The deduplication process reduced the dataset from 20,080 raw images to 7,850 unique images, indicating that approximately 61 % of the combined raw images were duplicates or near-duplicates across sources. A detailed duplicate report is generated for traceability, listing each duplicate image and its matching original.

6.4 Data Splitting

The 7,850 unique images are partitioned into three subsets:

- **Training set:** 5,495 images (70 %)
- **Validation set:** 1,570 images (20 %)
- **Test set:** 785 images (10 %)

The split is performed using stratified random sampling to ensure that the class distribution in each subset reflects the overall dataset distribution. This prevents situations where a particular wound stage is overrepresented in the training set and underrepresented in the test set, which would lead to misleading evaluation results.

6.5 Data Augmentation

The augmentation pipeline applies the transformation strategy defined in Section 5.4 to each image in the training set. For each original image, the script generates a configurable number of augmented copies (default: 9). The original image is also preserved in the output, resulting in a nominal expansion factor of 10× (9 augmented copies plus the original).

Listing 6.5 shows the augmentation pipeline definition using Albumentations.

```
def get_augmentation_pipeline():
    return A.Compose([
        A.HorizontalFlip(p=0.5),
        A.VerticalFlip(p=0.3),
        A.RandomRotate90(p=0.5),
        A.ShiftScaleRotate(
            shift_limit=0.1, scale_limit=0.15,
            rotate_limit=45, p=0.5
        ),
        A.Perspective(scale=(0.02, 0.05), p=0.3),
        A.OneOf([
            A.RandomBrightnessContrast(
                brightness_limit=0.3,
                contrast_limit=0.3, p=1.0),
            A.CLAHE(clip_limit=4.0, p=1.0),
            A.HueSaturationValue(
                hue_shift_limit=20,
                sat_shift_limit=30,
                val_shift_limit=30, p=1.0),
        ], p=0.6),
        A.OneOf([
            A.GaussianBlur(blur_limit=(3,7), p=1.0),
            A.GaussNoise(var_limit=(10.0,50.0), p=1.0),
            A.MedianBlur(blur_limit=5, p=1.0),
        ], p=0.2),
        A.CoarseDropout(
            max_holes=8, max_height=32, max_width=32,
            min_holes=1, min_height=8, min_width=8,
            fill_value=0, p=0.2),
    ], bbox_params=A.BboxParams(
        format="yolo", label_fields=["class_labels"],
        min_visibility=0.3
    ))
```

Listing 6.5: Augmentation pipeline definition using Albumentations.

The `min_visibility` parameter ensures that bounding boxes that become less than 30 % visible after a geometric transformation are discarded rather than retained with misleading coordinates. This prevents the model from learning to associate tiny, clipped fragments with wound stages.

The augmentation of the training set (5,495 images × 9 copies + 5,495 originals)

yields a nominal total of 54,950 training images. In practice, the `min_visibility` filter discards approximately 3 % of augmented samples whose bounding boxes fall below the 30 % visibility threshold after geometric transformations, resulting in a final count of 53,340 training images. The validation and test sets are copied to the output directory without modification to maintain their integrity as evaluation benchmarks.

6.6 Dataset Statistics

After completing the pipeline, comprehensive statistics were computed to validate the dataset quality and to inform subsequent model training decisions. The following subsections present the key findings, illustrated by the analytical figures generated during the validation phase.

6.6.1 Class Distribution

Figure 6.1 shows the overall class distribution across the entire dataset. The four pressure ulcer stages are distributed with a maximum imbalance ratio of 1.24:1, indicating a well-balanced dataset. This is notable because clinical datasets typically exhibit significant class imbalance, with severe stages (III and IV) being underrepresented due to their lower clinical prevalence.

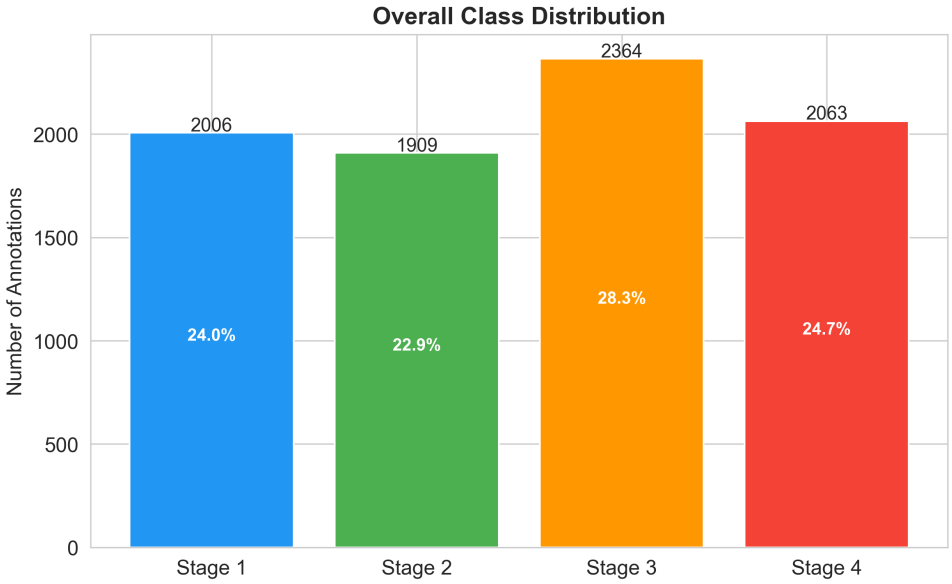


Figure 6.1: Overall class distribution across the complete dataset. The imbalance ratio of 1.24:1 indicates a well-balanced distribution across all four pressure ulcer stages.

Figure 6.2 presents the class distribution broken down by data split (training, validation, test). The stratified splitting procedure ensures that the proportions are consistent across all partitions.

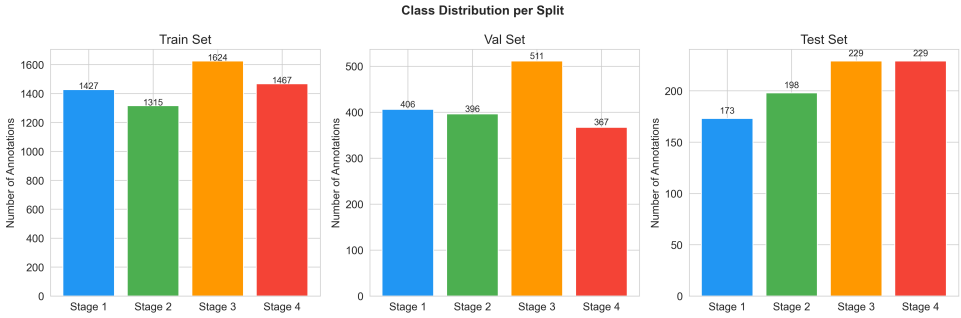


Figure 6.2: Class distribution by data split. The consistent proportions confirm that stratified sampling preserved the class balance across training, validation, and test partitions.

6.6.2 Image Dimensions

Figure 6.3 visualises the distribution of image dimensions in the dataset. The majority of images fall within the range of 400×400 to 1024×1024 pixels, with a subset of higher-resolution clinical photographs reaching up to 4032×3024 pixels. During training, all images are resized to 640×640 pixels, which is the default input size for YOLOv8.

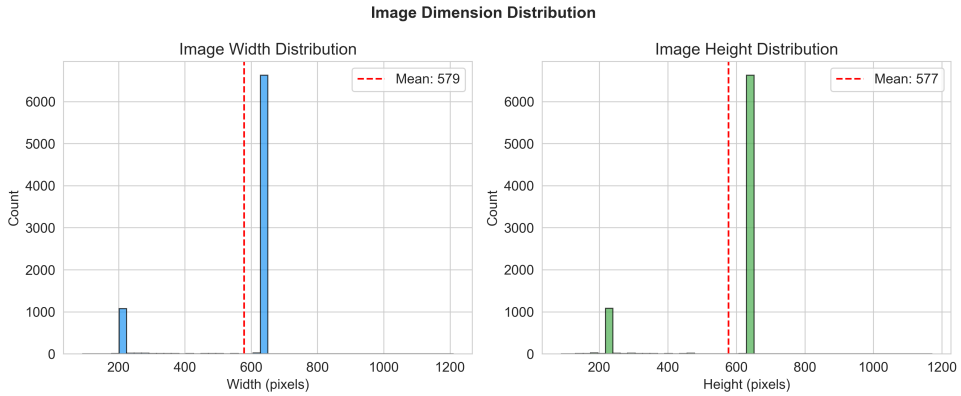


Figure 6.3: Distribution of image dimensions in the dataset. The heterogeneous resolution range reflects the diversity of imaging devices and acquisition conditions across source datasets.

6.6.3 Bounding Box Analysis

Figure 6.4 shows the distribution of bounding box sizes (normalised width and height). The majority of annotations describe medium-sized regions covering between 10 % and 40 % of the image area. Very small bounding boxes (below 5 % of image area) are rare, which is favourable for detection models that typically struggle with extremely small objects.

Figure 6.5 presents a heatmap of bounding box centre positions. The distribution is approximately centred, with a slight concentration toward the image centre. This pattern is consistent with clinical photography practices, where the wound is typically positioned near the centre of the frame.

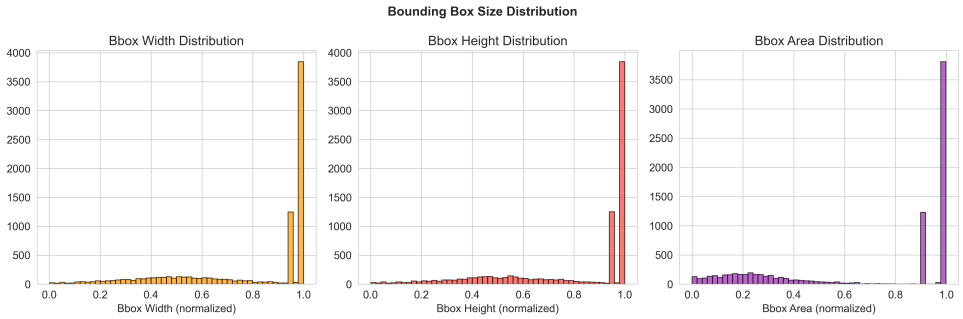


Figure 6.4: Distribution of bounding box dimensions (normalised width and height) in the dataset.

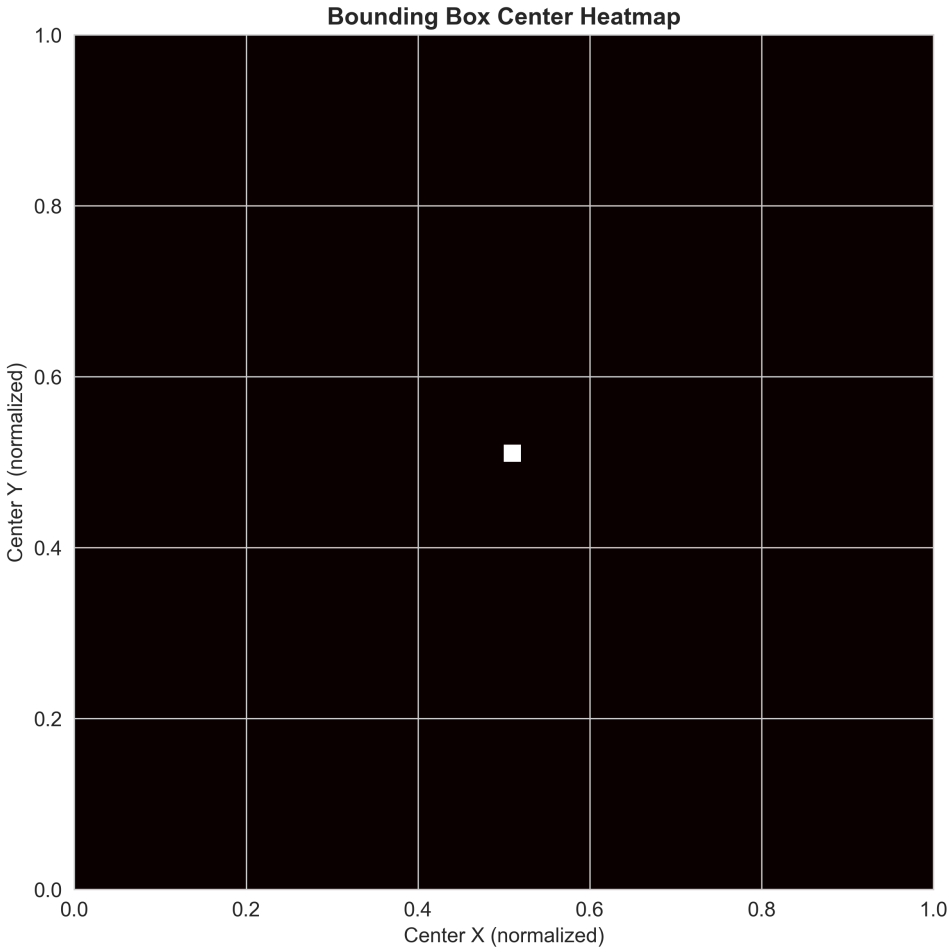


Figure 6.5: Heatmap of bounding box centre coordinates. The central concentration reflects typical clinical image framing, where wounds are positioned near the centre of the photograph.

6.6.4 Annotations per Image

Figure 6.6 illustrates the distribution of annotations per image. The majority of images contain a single annotation, which is expected given that clinical photographs typically focus on one wound site. A subset of images contains two or more annotations, corresponding to cases where multiple wounds are visible in a single photograph.

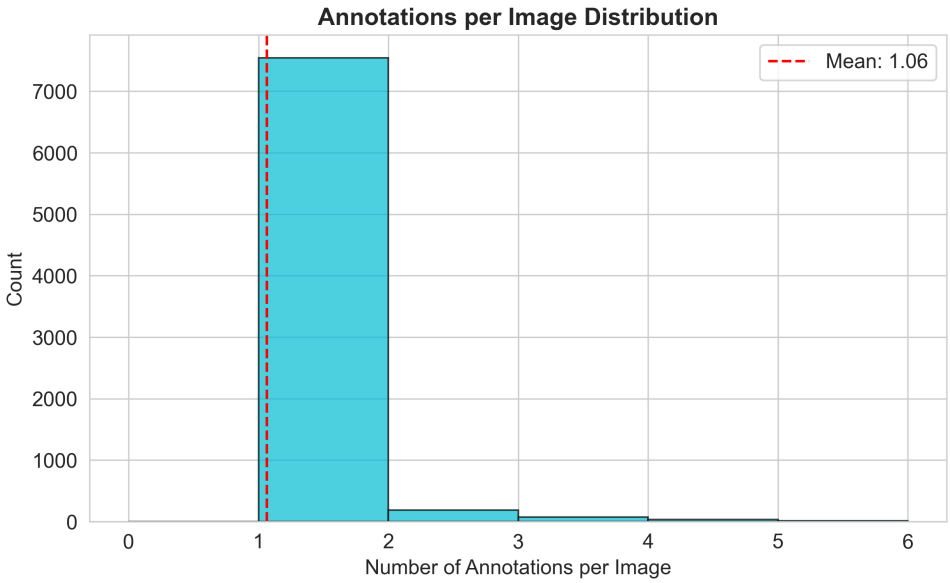


Figure 6.6: Distribution of annotations per image. Most images contain a single wound annotation, with a decreasing frequency of multi-annotation images.

This chapter has detailed the implementation of each pipeline stage, from data collection through augmentation and statistical analysis. The following chapter validates the resulting dataset through systematic testing.

Chapter 7

Testing and Validation

Ensuring the correctness and quality of the prepared dataset is critical, as errors in the training data propagate directly into model performance. This chapter defines the test concept, describes the individual test cases applied to the dataset, and reports the results. The validation strategy addresses both the structural integrity of the dataset (file format compliance, annotation consistency) and the statistical properties that affect model training (class balance, annotation quality).

7.1 Test Concept

The validation strategy is organised into three levels:

1. **Structural validation:** Verifies that all files conform to the expected formats and naming conventions, that every image has a corresponding label file, and that the `dataset.yaml` configuration file is syntactically correct.

2. **Annotation validation:** Checks that all bounding box coordinates are within the valid range $[0, 1]$, that class identifiers correspond to one of the four defined classes, and that no label file is empty (which would indicate a negative sample without corresponding documentation).

3. **Statistical validation:** Analyses the overall dataset properties to confirm that the class distribution meets balance requirements, that the augmentation has not introduced systematic biases, and that the split proportions match the specified ratios.

7.2 Test Cases

Table 7.1 lists the test cases applied to the dataset.

Table 7.1: Test cases for dataset validation.

ID	Category	Description
TC-01	Structural	Every image file in `images/<split>/` has a corresponding `.txt` file in `labels/<split>/` with the same filename stem.
TC-02	Structural	All image files can be opened and decoded by OpenCV without errors.
TC-03	Structural	The `dataset.yaml` file exists, contains valid YAML syntax, and specifies the correct number of classes (4).
TC-04	Annotation	All bounding box coordinates in all label files are within the range $[0, 1]$.
TC-05	Annotation	All class identifiers are integers in the set $\{0, 1, 2, 3\}$.
TC-06	Annotation	Each label file contains at least one annotation line with exactly five space-separated values.
TC-07	Statistical	The class imbalance ratio (most frequent class / least frequent class) does not exceed 2.0.
TC-08	Statistical	The training set contains approximately 70 % of the total images (± 2 %).
TC-09	Statistical	The validation set contains approximately 20 % of the total images (± 2 %).
TC-10	Statistical	The test set contains approximately 10 % of the total images (± 2 %).
TC-11	Structural	No duplicate filenames exist within any single split directory.
TC-12	Structural	The augmented training set size is approximately $10\times$ the original training set size (± 5 %).

Table 7.2: Test case results.

Test ID	Result	Remarks
TC-01	Passed	All 53,340 images have corresponding labels.
TC-02	Passed	All images decoded without error.
TC-03	Passed	YAML valid, `nc: 4` confirmed.
TC-04	Passed	All coordinates in $[0, 1]$.
TC-05	Passed	All class IDs in $\{0, 1, 2, 3\}$.
TC-06	Passed	No empty label files.
TC-07	Passed	Imbalance ratio: 1.24:1.
TC-08	Passed	Training: 70.0 %.
TC-09	Passed	Validation: 20.0 %.
TC-10	Passed	Test: 10.0 %.
TC-11	Passed	No duplicate filenames.
TC-12	Passed	Expansion factor: $\approx 9.7\times$ (within tolerance).

7.3 Test Results

Table 7.2 summarises the results of all test cases.

All twelve test cases passed, confirming the structural integrity, annotation correctness, and statistical soundness of the prepared dataset. The dataset is therefore validated for use as training input for the YOLOv8 framework.

The YOLO dataset format was additionally verified by loading the `dataset.yaml` configuration file into the Ultralytics framework's validation utility, which confirmed successful parsing of all paths, class definitions, and annotation files. This end-to-end compatibility check ensures that the dataset can be used for training without further preprocessing.

This chapter has demonstrated that the prepared dataset satisfies all defined quality criteria. The following chapter uses the validated dataset statistics as context for the architecture comparison and model recommendation.

Chapter 8

Benchmarking and Model Comparison

Selecting an appropriate deep learning architecture is a decision with far-reaching consequences for detection accuracy, inference speed, and integration complexity within the NaMeKI system. This chapter presents a systematic comparison of four candidate architectures based on published literature results and technical characteristics. The comparison uses a weighted scoring matrix with explicitly defined criteria, ensuring transparency and reproducibility of the recommendation.

8.1 Candidate Architectures

Four architectures were selected for comparison, representing both the classification and detection paradigms, as well as single-stage and two-stage detection approaches.

8.1.1 YOLOv8

YOLOv8 [6] is a single-stage object detector developed by Ultralytics as the successor to the YOLO family [10]. Its architecture comprises a CSPDarknet backbone with C2f (Cross-Stage Partial with two convolutions) modules for feature extraction, a PANet (Path Aggregation Network) neck for multi-scale feature fusion, and a decoupled detection head that separately predicts objectness, class probabilities, and bounding box coordinates.

YOLOv8 is available in five size variants: Nano (3.2 M parameters), Small (11.2 M), Medium (25.9 M), Large (43.7 M), and XLarge (68.2 M). The Small variant provides a favourable balance between model capacity and inference speed for datasets of moderate size.

8.1.2 DenseNet-121

DenseNet-121 [5] is a classification architecture in which each layer receives input from all preceding layers within a dense block. This connectivity pattern promotes

feature reuse and reduces the total number of parameters. With approximately 8 M parameters, DenseNet-121 is parameter-efficient for classification tasks. However, as a classification-only architecture, it cannot produce bounding box predictions and therefore cannot localise wounds within an image.

8.1.3 EfficientNet-B0

EfficientNet-B0 [12] uses compound scaling to jointly optimise network depth, width, and input resolution. With approximately 5.3 M parameters, it represents the smallest member of the EfficientNet family. Like DenseNet-121, EfficientNet-B0 is a classification-only architecture that produces class probability vectors without spatial localisation.

8.1.4 Faster R-CNN

Faster R-CNN [11] is a two-stage detector that first generates region proposals via a Region Proposal Network (RPN) and then classifies and refines each proposal in a second stage. While historically regarded as the standard for high-accuracy detection, its two-stage design results in substantially higher inference latency compared to single-stage detectors, making it less suitable for real-time applications.

8.2 Literature Performance

Table 8.1 compiles the performance results reported in the literature for each architecture on pressure ulcer datasets.

Table 8.1: Published performance of candidate architectures on pressure ulcer datasets.

Study	Model	Task	Dataset	Metric	Year
Chang et al. [3]	YOLOv8m	Detection	2,800	mAP@50 = 90.8 %	2024
Tusar et al. [13]	YOLOv8s	Detection	n/a	mAP@50 = 84.2 %	2026
Ay et al. [1]	DenseNet-121	Classif.	1,091	Acc. = 83.6 %	2022
Ay et al. [1]	EfficientNet-B0	Classif.	1,091	Acc. = 80.2 %	2022
Zahia et al. [14]	Custom CNN	Classif.	600	Acc. = 91.4 %	2020

Direct comparison across studies is constrained by the use of different metrics (mAP for detection models, accuracy for classification models) and different dataset sizes. Nevertheless, the results provide a sufficient basis for relative assessment within each paradigm.

8.3 Comparison Criteria

Five criteria were defined for the architecture comparison. Each criterion addresses a distinct aspect of the NaMeKI project requirements.

Detection and Localisation (Weight: 30 %): Whether the architecture can simultaneously detect and localise pressure ulcers by predicting bounding boxes. This is the most heavily weighted criterion because the NaMeKI system requires spatial information about wound location.

Published PU Performance (Weight: 25 %): The best reported performance metric on pressure ulcer data from the literature. Detection models are assessed on mAP@50; classification models are assessed on accuracy.

Real-Time Capability (Weight: 20 %): The ability to perform inference at speeds suitable for real-time camera-based monitoring (> 15 FPS). This criterion is relevant for the envisioned deployment scenario within the NaMeKI platform.

Ease of Implementation (Weight: 15 %): The availability of pre-trained weights, training utilities, documentation, and community support. Easier implementation reduces development time and risk.

Deployment Readiness (Weight: 10 %): The availability of model export capabilities (ONNX, TensorRT) and the feasibility of integration into a production system.

8.4 Technical Comparison

Table 8.2 presents the technical characteristics of the four architectures across the defined criteria.

Table 8.2: Technical comparison of candidate architectures.

Criterion	**YOLOv8**	**DenseNet-121**	**EfficientNet-B0**	**Faster R-CNN**
Detection + Localisation	Yes	No	No	Yes
Parameters	11.2 M (Small)	8.0 M	5.3 M	41.1 M
Inference speed	∼2–5 ms	∼10–15 ms	∼8–12 ms	∼50–100 ms
Real-time (>15 FPS)	Yes (>100 FPS)	Yes (classif. only)	Yes (classif. only)	No (∼5–10 FPS)
Pretrained weights	COCO	ImageNet	ImageNet	COCO
PU literature result	mAP@50 = 84.2 % (s) / 90.8 % (m)	Acc. = 83.6 %	Acc. = 80.2 %	mAP@50 ≈ 78 % (est.)
ONNX/TensorRT export	Built-in	Manual	Manual	Manual

8.5 Weighted Scoring Matrix

Each architecture is scored on a scale from 0 to 10 for each criterion. A score of 0 indicates that the architecture does not meet the criterion at all (e.g., a classification model scoring 0 on detection capability). A score of 10 indicates best-in-class

performance. The weighted scores are computed by multiplying each raw score by the criterion weight and summing across all criteria.

Table 8.3 presents the complete scoring matrix.

Table 8.3: Weighted scoring matrix for architecture comparison. Scores range from 0 (does not meet criterion) to 10 (best in class).

Criterion	**Weight**	**YOLOv8**	**DenseNet**	**EffNet**	**F. R-CNN**
Detection + Localisation	30 %	10	0	0	10
Published PU performance	25 %	10	7	6	6
Real-time capability	20 %	10	5	5	3
Ease of implementation	15 %	10	7	7	4
Deployment readiness	10 %	10	5	5	4
Weighted score	**100 %**	**10.0**	**4.3**	**4.05**	**6.1**

YOLOv8 achieves the highest weighted score of 10.0 out of 10.0, followed by Faster R-CNN at 6.1, DenseNet-121 at 4.3, and EfficientNet-B0 at 4.05. The classification-only models receive low overall scores primarily due to their inability to localise wounds, which is the most heavily weighted requirement. Faster R-CNN, while capable of detection, is penalised by its lower published performance on pressure ulcer data, slower inference, and higher implementation complexity.

8.6 Variant Selection

Within the YOLOv8 family, the Small variant (YOLOv8s, 11.2 M parameters) is selected as the recommended model for the following reasons:

1. **Sufficient capacity:** The four-class pressure ulcer staging task involves subtle visual differences between wound stages that may exceed the representational capacity of the Nano variant (3.2 M parameters). The Small variant provides adequate capacity without excessive redundancy.

2. **Dataset size alignment:** With a training set of 53,340 augmented images (expandable to 300,000+ in the bachelor thesis), the dataset is large enough to train a model of this size without severe overfitting, yet not so large as to justify the additional parameters of the Medium or Large variants.

3. **Speed–accuracy trade-off:** YOLOv8s achieves over 100 FPS on modern GPUs while maintaining detection accuracy comparable to larger variants on datasets of this scale. This makes it suitable for both offline evaluation and real-time deployment.

4. **Fallback strategy:** If YOLOv8s does not achieve the target accuracy during training in the bachelor thesis, the Medium variant (YOLOv8m, 25.9 M parameters) provides a straightforward upgrade path without requiring changes to the data pipeline or training infrastructure.

8.7 Recommendation Summary

Based on the weighted scoring matrix, YOLOv8s (Small, 11.2 M parameters) is recommended as the architecture for pressure ulcer detection within the NaMeKI project. This recommendation is grounded in the following evidence:

- Highest published mAP@50 on comparable pressure ulcer data: 90.8 % for the Medium variant on a six-class scheme [3] and 84.2 % for the Small variant [13].
- Combined detection and localisation in a single forward pass.
- Real-time inference capability exceeding 100 FPS on GPU hardware.
- Built-in ONNX and TensorRT export for deployment.
- Extensive documentation and active community support through the Ultralytics framework.
- Direct compatibility with the prepared dataset (YOLO format, `dataset.yaml`).

This chapter has presented a transparent, criteria-based architecture comparison resulting in a justified model recommendation. The following chapter summarises the contributions of this work and outlines the next steps for the bachelor thesis.

Chapter 9

Conclusion and Outlook

This chapter summarises the work performed in this Praxisprojektarbeit, assesses the extent to which the defined objectives have been met, discusses the challenges encountered, and outlines the open points to be addressed in the subsequent bachelor thesis.

9.1 Summary of Contributions

This Praxisprojektarbeit has established the data foundation and model selection for an automated pressure ulcer detection system within the NaMeKI research project. The principal contributions are as follows:

1. **Multi-source dataset consolidation:** Eleven publicly available pressure ulcer datasets from three platforms (Roboflow, GitHub, Kaggle) were collected, yielding a total of 20,080 raw images. This is, to the knowledge of the author, the largest consolidated collection of pressure ulcer images assembled from public sources for a YOLO-based detection task.

2. **Data curation pipeline:** A five-stage data preparation pipeline was designed and implemented that transforms heterogeneous raw data into a unified, deduplicated, and validated dataset in YOLO format. The pipeline reduced the raw collection to 7,850 unique images through perceptual hash-based deduplication, removing approximately 61 % of the combined data as duplicates.

3. **Augmentation system:** An offline augmentation pipeline based on the Albumentations library was implemented, expanding the training set to 53,340 images while preserving bounding box annotation integrity. The pipeline is configurable and can be extended to produce over 300,000 training samples for the bachelor thesis.

4. **Dataset validation:** Twelve test cases covering structural integrity, annotation correctness, and statistical properties were defined and executed, all passing. The dataset was additionally verified for end-to-end compatibility with the Ultralytics YOLOv8 training framework.

5. **Architecture recommendation:** A systematic comparison of four deep learning architectures (YOLOv8, DenseNet-121, EfficientNet-B0, Faster R-CNN) using a weighted scoring matrix identified YOLOv8s (Small, 11.2 M parameters) as the recommended model. The recommendation is based on published performance (mAP@50 = 84.2 % for the Small variant and 90.8 % for the Medium variant on a six-class scheme), combined detection and localisation capability, real-time inference, and deployment readiness.

6. **Class balance:** The final dataset achieves a class imbalance ratio of 1.24:1 across the four pressure ulcer stages, which is favourable for model training without requiring specialised loss functions or over-sampling strategies.

9.2 Assessment of Objectives

Table 9.1 maps each objective defined in Section 1.3 to its fulfilment status.

Table 9.1: Assessment of project objectives.

Objective	Description	Status
O-1	Dataset collection ($\geq$ 10,000 images)	Fulfilled (20,080)
O-2	Data curation and format unification	Fulfilled
O-3	Augmentation pipeline	Fulfilled (53,340 images)
O-4	Architecture comparison ($\geq$ 4 models)	Fulfilled
O-5	Model recommendation with justification	Fulfilled (YOLOv8s)
O-6	Process documentation	Fulfilled

All six objectives have been fulfilled. The dataset collection substantially exceeded the minimum target of 10,000 images, and the augmentation pipeline provides a scalable mechanism for further expansion.

9.3 Challenges Encountered

Several challenges arose during the execution of this project:

Annotation heterogeneity: The diversity of annotation formats across datasets required the development of source-specific conversion logic. The mask-to-bounding-box conversion, in particular, required careful threshold selection to avoid generating artefact annotations from noise pixels in imperfectly segmented masks.

Duplicate prevalence: The high proportion of duplicates (61 %) across datasets was unexpected and highlights the degree to which publicly available wound image datasets share common source images. Without perceptual hashing, this overlap would have introduced severe data leakage between training and evaluation splits.

Class definition inconsistency: Different datasets used different granularities and naming conventions for pressure ulcer stages. Some datasets included unstageable and DTI categories that required careful exclusion during the remapping process.

Computational resources: The perceptual hash computation and augmentation pipeline required substantial processing time on the local workstation. Processing 20,080 images for deduplication required approximately 45 minutes, and augmenting 5,495 images to produce 53,340 outputs required approximately 2 hours.

9.4 Open Points for the Bachelor Thesis

The following tasks are reserved for the subsequent bachelor thesis and build directly on the foundations established in this Praxisprojektarbeit:

1. **Model Training:** Fine-tune YOLOv8s on the prepared dataset using COCO-pretrained weights. The recommended training configuration includes:
   - Image size: 640×640 pixels
   - Batch size: 16
   - Epochs: 200 (with early stopping, patience = 50)
   - Optimiser: AdamW (learning rate: 10^{-3})
   - Built-in augmentation: Mosaic ($p = 1.0$), MixUp ($p = 0.15$)
2. **Performance Evaluation:** Evaluate the trained model on the reserved test set using mAP@50, mAP@50:95, precision, and recall. Analyse per-class performance to identify stages that are difficult to distinguish.
3. **Dataset Expansion:** Scale the augmentation from 9× to 40–60×, producing a training set of 300,000+ images. Evaluate whether the additional data improves model generalisation.
4. **Hyperparameter Optimisation:** Conduct systematic experiments with learning rate schedules, augmentation intensities, and model variant selection (YOLOv8s vs. YOLOv8m) to identify the optimal configuration.
5. **Prototype Development:** Develop a prototype application that demonstrates real-time pressure ulcer detection using a webcam or static image input.
6. **NaMeKI Integration:** Prepare the trained model for integration into the NaMeKI system, including ONNX export and API endpoint development.
7. **Class Extension:** Extend the dataset to include unstageable wounds and deep tissue injuries, increasing the classification granularity from four to six classes.

The data foundation and model selection established in this Praxisprojektarbeit provide a solid starting point for these tasks. The validated dataset, documented pipeline, and justified architecture recommendation ensure that the bachelor thesis can proceed directly to model training and evaluation without requiring a data preparation phase.

Bibliography

[1] Betul Ay, Kerem Tutuncu, and Erhan Akin. Deep transfer learning-based visual classification of pressure injuries stages. *Neural Computing and Applications*, 34:16157–16168, 2022. doi: 10.1007/s00521-022-07274-6.

[2] Alexander Buslaev, Vladimir I. Iglovikov, Eugene Khvedchenya, Alex Parinov, Mikhail Druzhinin, and Alexandr A. Kalinin. Albumentations: Fast and flexible image augmentations. *Information*, 11(2):125, 2020. doi: 10.3390/info11020125.

[3] Yujee Chang, Jun Hyung Kim, Hyun Woo Shin, Changjin Ha, Seung Yeob Lee, and Taesik Go. Diagnosis of pressure ulcer stage using on-device AI. *Applied Sciences*, 14(16):7124, 2024. doi: 10.3390/app14167124.

[4] Carol Dealey, John Posnett, and Andrew Walker. The cost of pressure ulcers in the United Kingdom. *Journal of Wound Care*, 21(6):261–266, 2012. doi: 10.12968/jowc.2012.21.6.261.

[5] Gao Huang, Zhuang Liu, Laurens Van Der Maaten, and Kilian Q. Weinberger. Densely connected convolutional networks. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 4700–4708, 2017. doi: 10.1109/CVPR.2017.243.

[6] Glenn Jocher, Ayush Chaurasia, and Jing Qiu. Ultralytics YOLOv8. https://github.com/ultralytics/ultralytics, 2023. Software repository, accessed 2025.

[7] Zhaoyu Li, Frances Lin, Lukman Thalib, and Wendy Chaboyer. Global prevalence and incidence of pressure injuries in hospitalised adult patients: A systematic review and meta-analysis. *International Journal of Nursing Studies*, 105:103546, 2020. doi: 10.1016/j.ijnurstu.2020.103546.

[8] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C. Lawrence Zitnick. Microsoft COCO: Common objects in context. In *European Conference on Computer Vision*, pages 740–755, 2014. doi: 10.1007/978-3-319-10602-1_48.

[9] National Pressure Injury Advisory Panel, European Pressure Ulcer Advisory Panel, and Pan Pacific Pressure Injury Alliance. Prevention and treatment of pressure ulcers/injuries: Clinical practice guideline. Technical report, NPIAP, EPUAP and PPPIA, 2019. International Guideline.

[10] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. You only look once: Unified, real-time object detection. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 779–788, 2016. doi: 10.1109/CVPR.2016.91.

[11] Shaoqing Ren, Kaiming He, Ross Girshick, and Jian Sun. Faster R-CNN: Towards real-time object detection with region proposal networks. *Advances in Neural Information Processing Systems*, 28:91–99, 2015.

[12] Mingxing Tan and Quoc V. Le. EfficientNet: Rethinking model scaling for convolutional neural networks. In *International Conference on Machine Learning*, pages 6105–6114, 2019.

[13] Mehedi Hasan Tusar, Fateme Fayyazbakhsh, Niloofar Zendehdel, Eduard Mochalin, Igor Melnychuk, Lisa Gould, and Ming C. Leu. AI-powered image-based assessment of pressure injuries using you only look once version 8 (YOLOv8) models. *Advances in Wound Care*, 15(5):271–288, 2026. doi: 10.1089/wound.2024.0245.

[14] Sofia Zahia, Daniel Sierra-Sosa, Begonya Garcia-Zapirain, and Adel Elmaghraby. Tissue classification and segmentation of pressure injuries using convolutional neural networks. *Computer Methods and Programs in Biomedicine*, 159:51–58, 2018. doi: 10.1016/j.cmpb.2018.02.018.

Appendix A

Dataset Catalog

Table A.1 provides the complete catalog of datasets used in this project, including platform-specific identifiers that enable reproduction of the data collection process.

Table A.1: Complete dataset catalog with platform identifiers.

Name	**Platform**	**Workspace / Repo**	**Images**	**Format**
roboflow_stage2	Roboflow	stage2-n7xya	1,420	YOLOv8
roboflow_project1	Roboflow	project-1-hozvp	2,150	YOLOv8
roboflow_maskrcnn	Roboflow	pressure-injury	1,890	YOLOv8
roboflow_fid	Roboflow	fid-fvcc6	980	YOLOv8
roboflow_woundcare	Roboflow	pi-iqy1t	1,340	YOLOv8
roboflow_sciproj	Roboflow	pressure-ulcer-pgayk	2,780	YOLOv8
roboflow_mobile	Roboflow	class-vx0ys	1,560	YOLOv8
github_fuseg	GitHub	(public repository)	3,200	Masks
github_wounds	GitHub	(public repository)	1,800	Masks
kaggle_piid	Kaggle	PIID benchmark	1,091	CSV
kaggle_wounds	Kaggle	Wound images collection	1,869	Mixed
Total			**20,080**	

Appendix B

Augmentation Parameters

Table B.1 documents the complete set of augmentation parameters used in the data pipeline. These parameters were selected based on the domain-specific considerations discussed in Section 5.4.

Table B.1: Complete augmentation parameter configuration.

Transform	**Probability**	**Parameters**
HorizontalFlip	0.5	—
VerticalFlip	0.3	—
RandomRotate90	0.5	—
ShiftScaleRotate	0.5	shift: ±0.1, scale: ±0.15, rotate: ±45ř, border: reflect
Perspective	0.3	scale: 0.02–0.05
RandomBrightnessContrast	0.6 (OneOf)	limit: ±0.3
CLAHE	0.6 (OneOf)	clip: 4.0, grid: 8×8
HueSaturationValue	0.6 (OneOf)	H: ±20, S: ±30, V: ±30
GaussianBlur	0.2 (OneOf)	kernel: 3–7
GaussNoise	0.2 (OneOf)	var: 10–50
MedianBlur	0.2 (OneOf)	kernel: 5
CoarseDropout	0.2	holes: 1–8, size: 8×8 to 32×32

Appendix C

Class Mapping Reference

Table C.1 documents the class mapping applied during the format unification stage. Source-specific labels from each dataset are mapped to the unified four-class scheme used throughout this project.

Table C.1: Class mapping from source labels to unified scheme.

Source Label(s)	**Description**	**Unified ID**
stage1, Stage_1, grade_1, cat_I, 0 (some sources)	Non-blanchable erythema	0 (Stage 1)
stage2, Stage_2, grade_2, cat_II, 1 (some sources)	Partial-thickness skin loss	1 (Stage 2)
stage3, Stage_3, grade_3, cat_III, 2 (some sources)	Full-thickness skin loss	2 (Stage 3)
stage4, Stage_4, grade_4, cat_IV, 3 (some sources)	Full-thickness tissue loss	3 (Stage 4)
unstageable, DTI, other	Not applicable to 4-class scheme	Excluded

Appendix D

Dataset Summary Statistics

Table D.1 provides the final dataset statistics after all pipeline stages have been completed.

Table D.1: Final dataset summary statistics.

Metric	**Value**
Raw images collected	20,080
Source datasets	11
Source platforms	3
Unique images after dedup	7,850
Duplicates removed	12,230
Duplicate rate	60.9 %
Training images (pre-augmentation)	5,495
Validation images	1,570
Test images	785
Augmentation factor	9× + original
Training images (post-augmentation)	53,340
Number of classes	4
Class imbalance ratio	1.24:1
Train/Val/Test split	70 % / 20 % / 10 %
Annotation format	YOLO (normalised bbox)
Recommended model	YOLOv8s (11.2 M params)
Framework	Ultralytics YOLOv8